Step 1: Import Express

```
const express = require("express");
```

Why?

Express is a framework of Node.js.

It helps us to:

- Create a server
- Create APIs
- Handle requests from users

Think like this:

Node.js
|
▼
Express
|
Makes server creation easy

Without Express, writing a server is difficult.

Step 2: Create an Express App

```
const app = express();
```

Here we create an application.

This app is the main object.

All routes (APIs) will be created using this app.

Like:

```
app.get(...)
app.post(...)
app.listen(...)
```

Think of it like:

Express
|
Create App
|
Now app controls everything

Step 3: Middleware

```
app.use(express.json());
```

This line is very important.

Suppose a user sends this data:

```
{
"id":4,
"name":"Rohit",
"course":"CSE"
}
```

Without middleware

Server cannot understand JSON properly.

With middleware

```
JSON
  |
Converted into JavaScript Object
  |
Stored inside req.body
```

Now we can write

req.body

and get the user's data.

So remember:

express.json()

↓

Reads JSON data

↓

Stores it in req.body

Step 4: Student Data

```
const students = [
  { id: 1, name: "Mukund", course: "CSE" },
  { id: 2, name: "Rahul", course: "AI" },
  { id: 3, name: "Aman", course: "IT" }
];
```

This is not a real database.

It is only an array.

Think like this:

Students

↓

Mukund

↓

Rahul

↓

Aman

We are storing student information.

Each student has

- id
- name
- course

Step 5: Product Data

```
const products = [
...
];
```

Same thing.

Here we store product information.

Products

↓

Laptop

↓

Phone

↓

Mouse

Step 6: Home Route

```
app.get("/", (req, res) => {
   res.send("Welcome to Express.js Lab");
});
```

What is GET?

GET means

Fetch data

or

Read data

Here

"/"

means Home Page.

When someone opens

http://localhost:3000

Express runs this code.

Browser

↓

GET /

↓

Server

↓

Welcome to Express.js Lab

res.send()

means

Send response back to the user.

Step 7: Get All Students

```
app.get("/students", (req, res) => {
   res.json(students);
```

});

URL

localhost:3000/students

User sends

GET request

Server returns

All students

Here

res.json()

means

Send data in JSON format.

Output

```
[
{
"id":1,
"name":"Mukund"
},
{
"id":2,
"name":"Rahul"
}
]
```

Step 8: Get Student by ID

app.get("/students/:id",...)

Notice

:id

This is called a **Route Parameter**.

Example

localhost:3000/students/2

Here

2

becomes the value of

req.params.id

Next line

const id = parseInt(req.params.id);

Why?

URL always gives a string.

Example

"2"

But student id is a number.

So we convert

"2"

↓

2

using

parseInt()

Next

const student = students.find(s => s.id === id);

This searches inside the array.

Imagine

Student Array

↓

Check Student 1

↓

Check Student 2

↓

Found!

↓

Return Student

If found

res.json(student);

If not found

res.status(404).send("Student Not Found");

404 means

Requested data does not exist.

Step 9: Add New Student

app.post("/students",...)

POST means

Add new data

Suppose user sends

```
{
"id":4,
"name":"Rohit",
"course":"ECE"
}
```

This data comes inside

req.body

Then

students.push(req.body);

means

Take the new student

↓

Insert into students array

Finally

res.send("Student Added Successfully");

Response goes back to the user.

Product APIs

Exactly same logic.

Only difference

Instead of

students

we use

products

Get All Products

GET

↓

/products

↓

Return all products

Get Product by ID

GET

↓

/products/2

↓

Find Product

↓

Return Product

Add Product

POST

↓

Receive data

↓

req.body

↓

push()

↓

Product Added

Last Step

app.listen(3000, () => {

console.log("Server is running...");

});

This starts the server.

Computer

↓

Starts Express Server

↓

Port 3000

↓

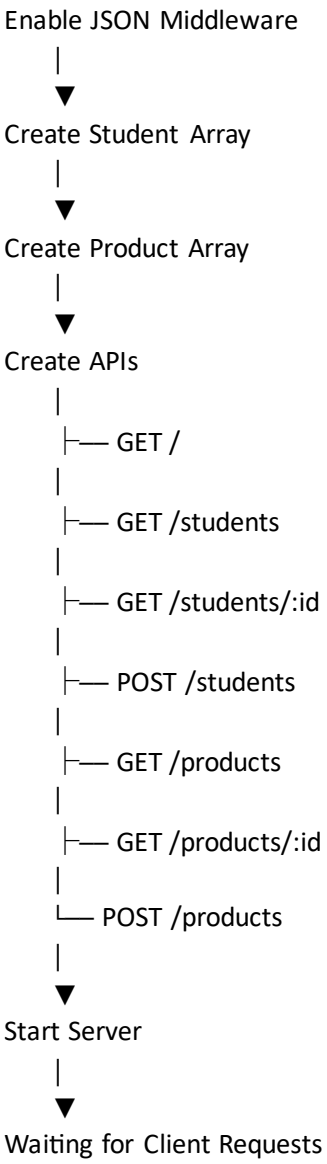
Ready to receive requests

Without this line

Server will never start.

Complete Flow of This Program

Start Program
|
▼
Import Express
|
▼
Create App
|
▼


Enable JSON Middleware
Create Student Array
Create Product Array
Create APIs
GET /
GET /students
GET /students/:id
POST /students
GET /products
GET /products/:id
POST /products
Start Server
Waiting for Client Requests